

Jordan Isaac

CS499

5/24/26

CS 499 Milestone Two Enhancement One: Software Engineering and Design

Artifact: CS465 Travlr Getaways

Brief Description of the Artifact

The artifact I selected for the software engineering and design enhancement is my CS465 Travlr Getaways full-stack web application. This project was originally created during the Full Stack Development course and was designed as a travel booking and trip management system.

The purpose of the application is to manage travel packages for both public users and authenticated administrators. The public-facing application allows users to browse available travel destinations and trip information, while the Angular administrative frontend allows authenticated users to create, update, and delete trip data through secured administrative functionality.

The application was developed using Angular, Node.js, Express, MongoDB, and Mongoose. The backend uses a RESTful API structure to communicate with the Angular frontend and MongoDB database. The application also utilizes JWT authentication middleware to protect administrative CRUD operations and secure sensitive backend routes.

The project follows separation of concerns by organizing the application into dedicated frontend components, backend routes, controllers, middleware, and database models. This

modular structure improves maintainability, scalability, readability, and long-term software management.

I originally created this artifact during CS465 because it represented one of my strongest examples of full-stack software engineering and backend API development.

Justification for Inclusion in the ePortfolio

I selected this artifact for inclusion in my ePortfolio because it demonstrates several important software engineering and full-stack development concepts. The application showcases my ability to design and develop a modular full-stack application that integrates frontend technologies, backend API services, middleware architecture, authentication security, and database management systems.

Several important software engineering concepts are demonstrated throughout the application, including RESTful API development, Angular single-page application architecture, reusable Angular components, CRUD functionality, middleware implementation, asynchronous database communication, and JWT authentication security.

One of the strongest aspects of the project is the separation of concerns between backend routes, controllers, middleware, database models, and Angular frontend components. This organizational structure demonstrates modular application architecture and improves maintainability, scalability, readability, and long-term software management.

During the code review process, I analyzed the application using a structured software engineering checklist to evaluate maintainability, defensive programming practices, security, code organization, and scalability.

The code review identified several areas for improvement. Some controllers contained duplicated validation logic, certain API routes lacked comprehensive input validation, backend error handling was inconsistent across controllers, and frontend validation was limited. While JWT authentication middleware had already been implemented successfully, additional security improvements and reusable middleware architecture would improve the long-term scalability and maintainability of the application.

To address these issues, I implemented several software engineering enhancements. One of the most significant enhancements involved implementing reusable validation middleware for trip creation and update operations. Previously, validation logic was duplicated or minimally implemented directly within backend controllers. I refactored this logic into reusable middleware components that validate incoming request data before database operations occur. The validation middleware now verifies required fields and validates numerical values such as trip pricing and trip length before requests are processed by the backend API.

I also implemented centralized error handling middleware to standardize backend error responses and improve maintainability and scalability throughout the API architecture. This enhancement reduced repeated error-handling logic across backend routes and improved consistency during backend failures.

Additional frontend enhancements were implemented within the Angular administrative interface. I improved frontend validation by validating required fields and numerical values

before requests are sent to the backend API. This created a layered validation architecture between the frontend and backend systems.

The application also continued using JWT authentication middleware to secure administrative CRUD routes while improving the authentication flow between the Angular frontend and Express backend.

These enhancements improved: maintainability, modularity, scalability, reusable architecture, defensive programming, frontend validation, backend consistency and authentication security

Course Outcomes

The enhancements completed during this milestone support several Computer Science program outcomes.

First, this enhancement demonstrates my ability to design, develop, and evaluate computing solutions using software engineering principles and modern full-stack development practices. The application demonstrates RESTful API development, modular backend architecture, Angular frontend development, middleware implementation, and scalable application organization.

Second, the enhancement demonstrates my ability to use well-founded and innovative tools, techniques, skills, and computing practices to improve maintainability, scalability, modularity, and reusable software architecture. The implementation of reusable middleware,

frontend validation systems, and centralized backend error handling demonstrates substantial progress toward professional software engineering practices.

Third, the enhancement demonstrates progress toward developing a security mindset that anticipates vulnerabilities and improves defensive programming practices. The application uses JWT authentication middleware to protect administrative functionality while implementing layered frontend and backend validation systems to reduce the likelihood of malformed or invalid requests reaching backend database operations.

The enhancement also demonstrates my ability to communicate technical solutions through modular code organization, reusable architecture, maintainable backend structure, and professionally organized frontend components.

Compared to my original Module One planning, the enhancement evolved beyond simple backend validation improvements into a larger software engineering enhancement involving reusable middleware architecture, frontend validation systems, centralized error handling, layered validation security, and improved authentication flow management.

While Milestone Two does not require demonstrating complete competency across every Computer Science program outcome, this enhancement demonstrates substantial progress toward competency in software engineering, secure backend architecture, scalable application design, defensive programming, modular development practices, and professional software organization.

Reflection on the Enhancement Process

Through the process of enhancing and modifying this artifact, I learned the importance of modular backend architecture and reusable software engineering practices.

One of the biggest lessons I learned involved the value of separating validation logic from backend controllers. Initially, much of the validation logic was duplicated across multiple API endpoints. Refactoring this functionality into reusable middleware significantly improved maintainability, backend organization, and scalability.

I also learned the importance of layered validation architecture within full-stack applications. Implementing validation within both the Angular frontend and Express backend improved both user experience and backend security while reducing invalid API requests.

Another important lesson involved centralized error handling. Prior to enhancement, backend errors were handled inconsistently across different routes and controllers. Implementing centralized error handling middleware improved consistency, scalability, debugging, and long-term maintainability throughout the backend API.

One challenge I encountered involved Angular standalone component configuration, dependency management, frontend routing behavior, and frontend validation integration. Resolving these issues strengthened my understanding of Angular architecture, standalone component management, and frontend dependency handling.

Another challenge involved ensuring that JWT authentication, middleware validation, CRUD operations, frontend validation, and backend routing all functioned correctly together without disrupting existing application functionality.

Overall, the enhancement process strengthened my understanding of: software engineering principles, reusable middleware architecture, full-stack application design, RESTful API development, Angular frontend architecture, defensive programming, scalable backend systems, centralized error handling, layered security architecture, professional software organization.

These enhancements demonstrate software engineering principles including modularity, maintainability, scalability, reusable architecture, defensive programming, secure backend design, and professional coding standards while demonstrating substantial progress toward several Computer Science program outcomes required for the ePortfolio assessment.

Enhancements:

Front End Validation

localhost:4200

Travlr GETAWAYS

Trips (curr

Log Out

localhost:4200 says
All fields are required.

OK

Add Trip

Add New Trip

Code

Name

Length

Start

Resort

Price

Image

Description

Add Trip Cancel



Beach Escape

7 nights

Resort: Sunny Shores Resort

Start: 2026-06-15

Price: 1299

Negative Length Test

localhost:4200

localhost:4200 says
Trip length must be greater than zero.

OK

TravlR
GETAWAYS

Trips (curr

Log Out

Add Trip

Add New Trip

Code TEST101

Name CS499 TEST TRIP

Length -3

Start June 2026

Resort Test Resort

Price 1000

Image /images/beach.jpg

Test description

Description

Add Trip Cancel



Beach Escape

7 nights

Resort: Sunny Shores Resort

Start: 2026-06-15

Price: 1299

Negative Price Test

localhost:4200

localhost:4200 says
Trip price must be greater than zero.
OK

TravlR
GETAWAYS

Trips (current)

[Log Out](#)

[Add Trip](#)

Add New Trip

Code: TEST101
Name: CS499 TEST TRIP
Length: 5
Start: June 2026
Resort: Test Resort
Price: -50
Image: /images/beach.jpg
Description: Test description

[Add Trip](#) [Cancel](#)



Beach Escape

7 nights

Resort: Sunny Shores Resort

Start: 2026-06-15

Price: 1299

Valid New Trip

localhost:4200/trips

Imported from Chr... SNHU Links DSA TV App to ESS and Subbing Footybite Original ... Planning Center H... Other favorites



Unique Test
7 Days
Resort: Castle Resort
Start: 4/25/26
Price: 1299
Test for CRUD.

Edit Trip Delete Trip



CS499 TEST TRIP
5
Resort: Test Resort
Start: June 2026
Price: 1000
Test description

Edit Trip Delete Trip